

Sistema TWSTFT — ROA

Guía de instalación y referencia de programas

Real Instituto y Observatorio de la Armada

Versión del documento: 1.1

Programas: lee_satres.py v0.5 · procesa_sesion.py v1.7 · calcula_diff.py v1.3 · envia_zabbix.py v0.1

Índice

1. Descripción del sistema
 2. Creación del usuario tw
 3. Instalación del sistema
 - - 3.1 Instalar dependencias del sistema
 - - 3.2 Clonar el repositorio
 - - 3.3 Crear el entorno virtual
 - - 3.4 Crear la estructura de directorios
 - - 3.5 Abrir el puerto 3020/UDP en el firewall
 4. Estructura de directorios
 5. Configuración de twstft.ini
 6. Puesta en marcha de lee_satres.py
 7. Programación de procesa_sesion.py con cron
 8. Referencia de programas
 - - 8.1 lee_satres.py
 - - 8.2 procesa_sesion.py
 - - 8.3 calcula_diff.py
-

1. Descripción del sistema

El sistema TWSTFT (Two-Way Satellite Time and Frequency Transfer) del ROA realiza comparaciones de alta precisión entre relojes atómicos de distintos laboratorios de metrología temporal a través de satélites geoestacionarios, siguiendo la Recomendación UIT-R TF.1153-4.

El sistema está compuesto por los siguientes programas Python:

Programa	Versión	Función
lee_satres.py	0.5	Receptor UDP. Escucha el puerto 3020 y graba los mensajes del módem SATRE en ficheros raw diarios.

procesa_sesion.py	1.7	Procesador de sesiones. Lee los ficheros raw, calcula los ajustes cuadráticos con filtro 5-sigma y genera los ficheros ITU (.tw) y report (.rp).
calcula_diff.py	1.3	Calcula las diferencias de tiempo entre laboratorios a partir de los ficheros ITU y genera ficheros .dat y gráficas HTML y PNG.
envia_zabbix.py	0.1	Calcula las diferencias de tiempo y las envía a un servidor Zabbix via zabbix_sender para monitorización y visualización en Grafana.

El fichero `twstft.ini` es el fichero de configuración central compartido por `lee_satres.py` y `procesa_sesion.py`.

2. Creación del usuario tw

Todos los pasos de esta sección se ejecutan como **root** o con un usuario con sudo completo.

2.1 Crear el usuario

```
sudo useradd -m -s /bin/bash -c "TWSTFT system user" tw
sudo passwd tw
```

2.2 Añadir sudo con contraseña

```
sudo usermod -aG wheel tw
```

- En AlmaLinux el grupo `wheel` tiene acceso a sudo por defecto.
- Verificar que en `/etc/sudoers` esté activa la línea:
- `%wheel ALL=(ALL) ALL`

2.3 Habilitar el servicio systemd de usuario

Para que el servicio `systemd --user` de `lee_satres.py` arranque automáticamente sin que el usuario haya iniciado sesión:

```
sudo loginctl enable-linger tw
```

2.4 Verificar

```
id tw
# uid=1001(tw) gid=1001(tw) grupos=1001(tw),10(wheel)

loginctl show-user tw | grep Linger
# Linger=yes
```

3. Instalación del sistema

Iniciar sesión como usuario tw:

```
su - tw
```

3.1 Instalar dependencias del sistema

```
sudo dnf install -y git python3 python3-pip python3-virtualenv
```

3.2 Clonar el repositorio

```
cd /home/tw
git clone https://github.com/luisbat/twstft.git
cd twstft
chmod +x lee_satres.py procesa_sesion.py calcula_diff.py
```

3.3 Crear el entorno virtual e instalar dependencias Python

```
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install numpy pandas matplotlib plotly
deactivate
```

El entorno virtual queda en /home/tw/twstft/venv/.

3.4 Crear la estructura de directorios de datos

```
mkdir -p /home/tw/satres/log
mkdir -p /home/tw/ema
```

- El subdirectorio <unit_id> (p.ej. 448/) y sus subdirectorios raw/, itu/, rp/
- los crea automáticamente lee_satres.py y procesa_sesion.py en la primera ejecución.

3.5 Abrir el puerto 3020/UDP en el firewall

lee_satres.py escucha en el puerto UDP 3020. En AlmaLinux el firewall está activo por defecto y hay que abrir ese puerto:

```
sudo firewall-cmd --permanent --add-port=3020/udp
sudo firewall-cmd --reload
```

Verificar que el puerto está abierto:

```
sudo firewall-cmd --list-ports
# Debe aparecer: 3020/udp
```

4. Estructura de directorios

```
/home/tw/
twstft/      ← repositorio (scripts + configuración)
venv/       ← entorno virtual Python
```

```

lee_satres.py
procesa_sesion.py
calcula_diff.py
twstft.ini
lee_satres.service

satres/          ← todos los datos del sistema
<unit_id>/      ← p.ej. 448/
  raw/          ← ficheros raw diarios
    raw.YYYYYMMDD
  itu/          ← ficheros ITU generados
    twroa61.140
  rp/           ← ficheros report generados
    rproa61.140
  log/          ← logs de todos los programas y unidades
    lee_satres.YYYY-MM-DD.log
    procesa_sesion.YYYY-MM-DD.log

ema/             ← datos ambientales (independiente de unit_id)
  DDmmMAA.ema    ← puede ser un montaje NFS sobre este directorio

```

5. Configuración de twstft.ini

El fichero `twstft.ini` del repositorio contiene la configuración de referencia. Antes de poner el sistema en marcha hay que revisar y ajustar al menos las siguientes secciones:

5.1 Sección [ficheros]

```

[ficheros]
base      = /home/tw/satres      # directorio raíz de datos por unidad
raw_dir   = raw                 # subdirectorio para ficheros raw
ema_dir   = /home/tw/ema        # ruta absoluta a ficheros .ema (puede ser NFS)
itu_dir   = itu                 # subdirectorio para ficheros ITU
rp_dir    = rp                  # subdirectorio para ficheros report
log_dir   = /home/tw/satres/log # directorio de logs (compartido por todas las unidades)

```

5.2 Sección [itu]

```

[itu]
lab      = ROA                  # identificador del laboratorio (BIPM)
modem    = SATRE, 448           # tipo y número de serie del módem

```

■ El número al final de `modem` se usa como `unit_id` para los subdirectorios.

5.3 Sección [zabbix]

Configuración del servidor Zabbix para el envío de datos desde `envia_zabbix.py`:

```

[zabbix]
servidor   = zabbix.roa.es      # hostname o IP del servidor Zabbix
puerto    = 10051              # puerto Zabbix trapper (defecto 10051)
host       = TWSTFT             # nombre del host en Zabbix
item_prefix = twstft            # prefijo de los item keys

```

```
activo      = no          # si/no – activar envío a Zabbix
```

Los item keys se generan automáticamente como <prefix>.<loc>.<rem>, por ejemplo `twstft.roa01.ptb05`.

5.4 Parámetros de descarga en [lab XXXX]

Cada sección de laboratorio puede incluir la URL de descarga de sus ficheros ITU:

```
[lab PTB05]
...
itu_url    = ftp://usuario:password@ftp.ptb.de/pub/twstft
itu_mayus  = no      # si/no – nombre del fichero en mayúsculas
```

Cuando `calcula_diff.py` o `envia_zabbix.py` se ejecutan con `--config` y el laboratorio remoto no tiene directorio local especificado, descargan automáticamente el fichero ITU desde esta URL.

5.5 Verificar los laboratorios activos

Revisar las secciones `[lab xxxx]` y asegurarse de que los parámetros `prn`, `calr`, `ci` y `refdelay` están actualizados según la última calibración.

6. Puesta en marcha de `lee_satres.py`

`lee_satres.py` se ejecuta como un servicio `systemd --user` del usuario `tw`. Esto significa que el servicio corre sin privilegios de root y arranca automáticamente al iniciar el sistema gracias a `loginctl enable-linger`.

6.1 Instalar el fichero de servicio

```
mkdir -p ~/.config/systemd/user
cp /home/tw/twstft/lee_satres.service ~/.config/systemd/user/
```

El fichero `lee_satres.service` del repositorio tiene el siguiente contenido:

```
# Versión: 0.2
[Unit]
Description=SATRE UDP listener – lee_satres
After=network.target

[Service]
Type=simple
WorkingDirectory=/home/tw/twstft
ExecStart=/home/tw/twstft/venv/bin/python3 /home/tw/twstft/lee_satres.py \
    --config /home/tw/twstft/twstft.ini
Restart=on-failure
RestartSec=10
StandardOutput=null
StandardError=null
```

```
[Install]
WantedBy=default.target
```

6.2 Activar y arrancar el servicio

```
systemctl --user daemon-reload
systemctl --user enable lee_satres.service
systemctl --user start lee_satres.service
```

6.3 Verificar el estado

```
systemctl --user status lee_satres.service
```

La salida debe mostrar Active: active (running).

6.4 Consultar el log

```
tail -f /home/tw/satres/log/lee_satres.YYYY-MM-DD.log
```

O en modo depuración (sin lanzar el daemon, útil para pruebas):

```
/home/tw/twstft/venv/bin/python3 /home/tw/twstft/lee_satres.py \
--config /home/tw/twstft/twstft.ini --debug
```

6.5 Detener o reiniciar el servicio

```
systemctl --user stop lee_satres.service
systemctl --user restart lee_satres.service
```

7. Programación de procesa_sesion.py con cron

procesa_sesion.py se lanza con cron cada dos horas en los minutos pares más 5 minutos, procesando el raw del día anterior completo.

7.1 Editar el crontab del usuario tw

```
crontab -e
```

Añadir las siguientes líneas:

```
# A las 00:05 procesar el día anterior completo (última sesión ya terminada)
5 0 * * * /home/tw/twstft/venv/bin/python3 \
/home/tw/twstft/procesa_sesion.py \
--config /home/tw/twstft/twstft.ini \
--ayer

# A las 02:05, 04:05, ..., 22:05 procesar el día en curso
# (acumula las sesiones conforme van terminando)
5 2,4,6,8,10,12,14,16,18,20,22 * * * /home/tw/twstft/venv/bin/python3 \
/home/tw/twstft/procesa_sesion.py \
--config /home/tw/twstft/twstft.ini \
```

--hoy

- A las 00:05 el día anterior ya ha terminado — su última sesión
- (22:00-23:59) ha concluido — y se regenera el fichero ITU completo
- con --ayer. El resto de lanzamientos usan --hoy para ir
- acumulando las sesiones del día en curso conforme terminan.

7.2 Verificar el crontab

```
crontab -l
```

7.3 Consultar el log

```
tail -f /home/tw/satres/log/procesa_sesion.YYYY-MM-DD.log
```

8. Referencia de programas

8.1 lee_satres.py

Versión: 0.4

Receptor UDP que escucha el puerto configurado y graba los mensajes del módem SATRE en ficheros raw diarios. Se ejecuta como daemon con rotación automática de fichero al cambiar el día.

Estructura de salida:

<base>/<unit_id>/raw/raw.YYYYMMDD

Opciones:

Opción	Argumento	Descripción
--config	RUTA	Ruta al fichero twstft.ini. Defecto: twstft.ini en el mismo directorio del script.
--debug	—	Modo depuración: log detallado en consola, no daemoniza.
--version	—	Muestra la versión del programa y termina.

Ejemplos:

```
# Arrancar en modo daemon (uso normal)
python3 lee_satres.py --config /home/tw/twstft/twstft.ini
```

```
# Arrancar en modo depuración (para pruebas)
python3 lee_satres.py --config /home/tw/twstft/twstft.ini --debug
```

Parámetros leídos de twstft.ini:

Sección	Clave	Uso
---------	-------	-----

[network]	puerto	Puerto UDP de escucha (defecto: 3020).
[ficheros]	base	Directorio raíz de datos.
[ficheros]	raw_dir	Subdirectorio para ficheros raw (defecto: raw).

8.2 procesa_sesion.py

Versión: 1.7

Procesador de sesiones TWSTFT. Lee los ficheros raw del módem SATRE, calcula el ajuste cuadrático con filtro iterativo 5-sigma por slot de 3 minutos, e interpola los datos ambientales del fichero .ema. Genera simultáneamente el fichero ITU y el fichero report.

Estructura de salida:

```
<base>/<unit_id>/itu/twroa<MM>.<MMM> ← fichero ITU (Anexo 2, UIT-R TF.1153-4)
<base>/<unit_id>/rp/rproa<MM>.<MMM> ← fichero report
```

Opciones:

Opción	Argumento	Descripción
--config	RUTA	Ruta al fichero twstft.ini. Defecto: twstft.ini en el mismo directorio.
--hoy	—	Procesar el raw de hoy. Equivalente al comportamiento por defecto si no se especifica fecha.
--ayer	—	Procesar el raw de ayer. Opción recomendada para uso en cron.
--fecha	YYYYMMDD	Procesar el raw de una fecha específica.
--raw	RUTA	Ruta explícita a un fichero raw a procesar. Si se omite, se busca en <base>/<unit_id>/raw/raw.YYYYMMDD.
--debug	—	Log detallado en consola.
--version	—	Muestra la versión del programa y termina.

- **Prioridad de fecha:** --fecha > --ayer > --hoy > defecto (hoy).
- Siempre se procesa el día completo (sesiones 00:00 a 22:00).
- Si el fichero ITU ya existe, se borra y se regenera desde cero.

Ejemplos:

```
# Procesar el día de ayer (uso normal desde cron)
python3 procesa_sesion.py --config twstft.ini --ayer

# Procesar una fecha específica
python3 procesa_sesion.py --config twstft.ini --fecha 20260410

# Procesar un fichero raw explícito (para pruebas o reprocesado)
python3 procesa_sesion.py --config twstft.ini --raw /tmp/raw.20260410 --debug

# Procesar el día de hoy
python3 procesa_sesion.py --config twstft.ini --hoy
```

Parámetros leídos de twstft.ini:

Sección	Clave	Uso
[ficheros]	base, raw_dir, ema_dir, itu_dir, rp_dir	Rutas de entrada y salida.
[local]	par, impar	Laboratorio local para hora par e impar.
[itu]	lab, modem, ntl, ...	Metadatos del encabezamiento ITU.
[lab XXXX]	todos	Parámetros de cada laboratorio remoto.

Algoritmo de procesado por slot:

Cada sesión de 2 horas se divide en slots de 3 minutos:

- - **Minuto 0** del slot: enganche del receptor → datos descartados.
- - **Minutos 1-2:** hasta 120 muestras válidas.
- - Se descarta el slot si hay menos de 50 muestras antes del filtrado.
- - Ajuste cuadrático de grado 2 + filtro iterativo 5-sigma (σ muestral, $ddof=1$).
- - Se descarta el slot si quedan menos de 50 muestras tras el filtrado.
- - El punto de evaluación del polinomio es **t = 59.5 s**.

8.3 calcula_diff.py

Versión: 1.3

Calcula las diferencias de tiempo UTC(local) - UTC(lab_remoto) entre el laboratorio local y uno o varios laboratorios remotos, a partir de sus ficheros ITU. Genera ficheros .dat con las diferencias slot a slot y gráficas HTML (plotly) y PNG (matplotlib) con todos los laboratorios.

twstft.ini es opcional: si se proporciona con --config, permite la descarga automática de ficheros ITU remotos via FTP/HTTP usando los parámetros itu_url e itu_mayus de cada sección [lab XXXX].

Para cada par de laboratorios y cada día de la ventana:

1. Genera un fichero .dat con las diferencias slot a slot.
2. Genera una gráfica HTML interactiva (plotly) y una PNG (matplotlib) con todos los laboratorios en la misma figura.

Ecuación (UIT-R TF.1153-4, sección 8.2):

Para S=1 (calibrado):

$$\begin{aligned} \text{UTC}(1) - \text{UTC}(2) = & +0.5 \cdot [\text{TW}(1) + \text{ESDVAR}(1)] + \text{REFDELAY}(1) \\ & - 0.5 \cdot [\text{TW}(2) + \text{ESDVAR}(2)] - \text{REFDELAY}(2) \\ & + 0.5 \cdot [\text{CALR}(1,2) - \text{CALR}(2,1)] \end{aligned}$$

Para S=9 (no calibrado, resultado con desplazamiento desconocido K):

$$\begin{aligned} \text{UTC}(1) - \text{UTC}(2) + K = & +0.5 \cdot [\text{TW}(1) + \text{ESDVAR}(1)] + \text{REFDELAY}(1) \\ & - 0.5 \cdot [\text{TW}(2) + \text{ESDVAR}(2)] - \text{REFDELAY}(2) \end{aligned}$$

Si S del laboratorio local y del remoto no coinciden en un slot,
ese slot se descarta con un aviso.

Estructura de salida:

```
<salida>/diff_<local>_<remoto>_<MJD>.dat ← diferencias slot a slot
<salida>/diff_<local>_<MJD>_v<N>.html ← gráfica interactiva (plotly)
<salida>/diff_<local>_<MJD>_v<N>.png ← gráfica para informes (matplotlib)
```

Opciones:

Opción	Argumento	Descripción
--local	RUTA	Fichero ITU del laboratorio local del día más reciente. Obligatorio.
--remoto	LAB:DIR	Laboratorio remoto y directorio donde están sus ficheros ITU. Repetir para varios laboratorios. Al menos uno obligatorio.
--ventana	N	Número de días a procesar hacia atrás desde el fichero --local. Defecto: 1.
--salida	RUTA	Directorio de salida para .dat y gráficas. Defecto: directorio actual.
--config	RUTA	Ruta a twstft.ini. Opcional. Activa la descarga automática de ficheros ITU remotos via FTP/HTTP.
--debug	—	Log detallado en consola.
--version	—	Muestra la versión del programa y termina.

- Los nombres de los ficheros ITU remotos siguen el patrón
- tw<letras_lab><MM>.<MMM>, insensible a mayúsculas/minúsculas.
- Por ejemplo, PTB05 → twptb61.140 o TWPTB61.140.
- El directorio en --remoto es opcional si se usa --config con itu_url
- configurado en la sección [lab XXXX] correspondiente.

Ejemplos:

```
# Calcular diferencias ROA-PTB y ROA-SP para el día del fichero ITU
python3 calcula_diff.py \
  --local /home/tw/satres/448/itu/twroa61.199 \
  --remoto PTB05:/datos/ptb \
  --remoto SP01:/datos/sp

# Ventana de 5 días con directorio de salida explícito
python3 calcula_diff.py \
  --local /home/tw/satres/448/itu/twroa61.199 \
  --remoto PTB05:/datos/ptb \
  --remoto SP01:/datos/sp \
  --remoto USN001:/datos/usno \
  --ventana 5 \
  --salida /home/tw/satres/diff

# Calcular desde el punto de vista del PTB (PTB como laboratorio local)
python3 calcula_diff.py \
  --local /datos/ptb/twptb61.199 \
  --remoto ROA01:/home/tw/satres/448/itu \
  --remoto SP01:/datos/sp
```

Formato del fichero .dat:

```
* diff_roa01_ptb05_61199.dat
* UTC(ROA01) - UTC(PTB05)
```

```
* MJD: 61199 Slots: 12 Media: -2.102 ns sigma: 0.090 ns
*
* MJD STTIME DIFF_NS TW_A_NS TW_B_NS SMP_A SMP_B DRMS_A DRMS_B
* hhmmss ns ns ns
61199 001600 -2.087 +260359477.274 +260359970.698 120 120 0.564 0.493
61199 021600 -2.095 ...
...
```

8.4 envia_zabbix.py

Versión: 0.1

Calcula las diferencias de tiempo UTC(local) - UTC(lab_remoto) entre el laboratorio local y uno o varios laboratorios remotos, a partir de sus ficheros ITU, y envía los resultados a un servidor Zabbix vía zabbix_sender.

Cada slot se envía como un dato independiente con su **timestamp real** (derivado del MJD + STTIME), lo que permite a Zabbix y Grafana representar la serie temporal completa y configurar alertas sobre valores fuera de norma.

La configuración del servidor Zabbix se lee de la sección [zabbix] del fichero twstft.ini. El envío solo se realiza si activo = si.

Item key generado:

```
<item_prefix>.<loc_lower>.<rem_lower>
Ejemplo: twstft.roa01.ptb05
```

Requisito del sistema: zabbix_sender debe estar instalado:

```
sudo dnf install zabbix-sender
```

Opciones:

Opción	Argumento	Descripción
--local	RUTA	Fichero ITU del laboratorio local del día más reciente. Obligatorio.
--remoto	LAB[:DIR]	Laboratorio remoto y directorio de sus ficheros ITU. El directorio es opcional si itu_url está configurado en el ini. Repetir para varios labs. Al menos uno obligatorio.
--ventana	N	Número de días a procesar hacia atrás. Defecto: 1.
--config	RUTA	Ruta a twstft.ini. Necesario para leer la configuración Zabbix y para la descarga automática de ficheros remotos. Defecto: twstft.ini en el mismo directorio del script.
--debug	—	Log detallado en consola.
--version	—	Muestra la versión del programa y termina.

Ejemplos:

```
# Enviar diferencias ROA-PTB y ROA-SP del día actual
python3 envia_zabbix.py \
  --local /home/tw/satres/448/itu/twroa61.199 \
  --remoto PTB05:/home/tw/satres/itu_remoto/ptb \
  --remoto SP01:/home/tw/satres/itu_remoto/sp \
```

```
--config /home/tw/twstft/twstft.ini

# Con descarga automática via FTP (sin directorio local)
python3 envia_zabbix.py \
    --local /home/tw/satres/448/itu/twroa61.199 \
    --remoto PTB05 \
    --remoto SP01 \
    --config /home/tw/twstft/twstft.ini

# Ventana de 5 días desde cron
python3 envia_zabbix.py \
    --local /home/tw/satres/448/itu/twroa61.199 \
    --remoto PTB05 --remoto SP01 --remoto USN001 \
    --ventana 5 \
    --config /home/tw/twstft/twstft.ini
```

Ejemplo de entrada en crontab:

```
# Enviar a Zabbix a las 01:00 con los datos del día anterior
0 1 * * * /home/tw/twstft/venv/bin/python3 \
    /home/tw/twstft/envia_zabbix.py \
    --local /home/tw/satres/448/itu/${date -d yesterday +twroa%y.%j} \
    --remoto PTB05 --remoto SP01 \
    --config /home/tw/twstft/twstft.ini
```

Apéndice A — Verificación rápida del sistema

```
# 1. Comprobar que lee_satres está activo
systemctl --user status lee_satres.service

# 2. Comprobar que se están grabando datos raw
ls -lh /home/tw/satres/448/raw/

# 3. Procesar manualmente el día de ayer
python3 /home/tw/twstft/procesa_sesion.py \
    --config /home/tw/twstft/twstft.ini --ayer --debug

# 4. Verificar fichero ITU generado
ls -lh /home/tw/satres/448/itu/

# 5. Calcular diferencias con un laboratorio remoto
python3 /home/tw/twstft/calcula_diff.py \
    --local /home/tw/satres/448/itu/twroa61.199 \
    --remoto PTB05:/home/tw/satres/itu_remoto/ptb \
    --salida /tmp
```

Apéndice B — Dependencias Python

Paquete	Uso
numpy	Ajuste cuadrático, filtro 5-sigma, estadísticas
pandas	Lectura y procesado de datos ITU y raw
matplotlib	Gráficas PNG para informes

plotly	Gráficas HTML interactivas
--------	----------------------------

Instalación:

```
source /home/tw/twstft/venv/bin/activate
pip install numpy pandas matplotlib plotly
deactivate
```

Dependencias del sistema (adicionales a Python):

Paquete	Instalación	Necesario para
zabbix-sender	sudo dnf install zabbix-sender	envia_zabbix.py

Apéndice C — Formato del fichero raw

El fichero raw.YYYYMMDD contiene todos los mensajes UDP emitidos por el módem SATRE tal como llegan, sin procesar. Cada línea corresponde a un mensaje con un prefijo que indica su tipo:

Prefijo	Contenido
%0>	Sincronización. Contiene la fecha, hora del módem y unit_id. Se usa para la rotación diaria del fichero.
%Tx>	Datos de transmisión local.
%Rx1>	Datos de recepción del canal 1. Contiene 24 campos separados por ;: fecha, hora, unit_id, PRN del lab remoto, RTT, frecuencia Rx, potencia de señal y C/No.
%Rx2>, %Rx3>	Canales adicionales de recepción (no usados actualmente).

procesa_sesion.py usa exclusivamente las líneas %Rx1> en formato largo (L), que contienen el RTT completo necesario para el cálculo del TW. El fichero se rota automáticamente a medianoche — un fichero por día.

Ejemplo de línea %Rx1> en formato largo:

```
%Rx1>2026/04/10;00:01:00;448;00000;INT;L_0;L;yynxy;T;3;10;733;262383479.805; \
0.000;0.000;0.494;0.000;0.000;0.000;0.000;0.0000;70947202.453;-44.4;53.9
```

Los campos relevantes para procesa_sesion.py son:

Campo	Índice	Contenido
Fecha	0	YYYY/MM/DD
Hora	1	HH:MM:SS
Unit ID	2	Identificador del módem local
Formato	6	L = largo, S = corto
PRN	10	Código PN del laboratorio remoto
RTT	12	Round-trip time bruto [ns]
Frecuencia Rx	21	Frecuencia real de recepción [Hz]

Potencia	22	Potencia de señal [dBm]
C/No	23	Relación portadora/ruido [dBHz]

Documento generado para el sistema TWSTFT del ROA — Real Instituto y Observatorio de la Armada